



# ARTIFACTORY

## FOR COMPLETE BEGINNERS



**STORE. MANAGE. SECURE. DELIVER.**  
**ARTIFACTS THE RIGHT WAY**



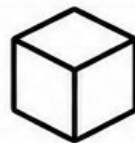
CODE



BUILD



ARTIFACTORY



ARTIFACT



DEPLOY



**CENTRALIZED  
REPOSITORIES**



**SECURE  
ARTIFACTS**



**ACCELERATE  
DELIVERY**



**TRACK &  
CONTROL**



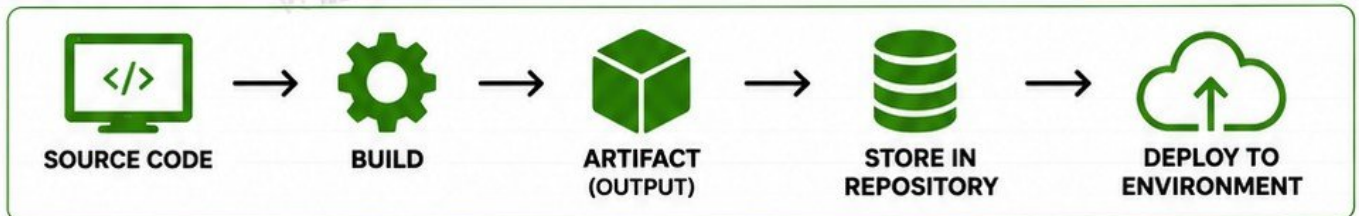
**IMPROVE  
RELIABILITY**





# 1. WHY ARTIFACT REPOSITORIES EXIST

## ► WHAT HAPPENS AFTER SOURCE CODE IS BUILT



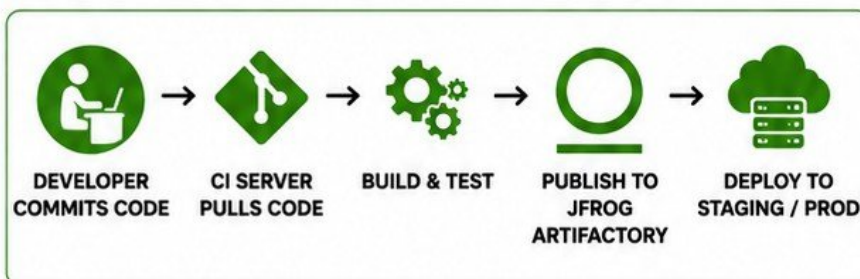
## ► SOURCE CODE VS BUILD ARTIFACTS

| SOURCE CODE        | BUILD ARTIFACT                |
|--------------------|-------------------------------|
| Human readable     | Binary / compiled             |
| Changes frequently | Immutable                     |
| Stored in Git      | Stored in Artifact Repository |
| Many variations    | One version is exact          |
| Needs to be built  | Ready to deploy / use         |

## ► WHY TEAMS SHOULD NOT REBUILD THE SAME ARTIFACT REPEATEDLY

- ✓ Inconsistent builds cause production issues
- ✓ Different environments may produce different outputs
- ✓ Rebuilding wastes time and compute resources
- ✓ Harder to trace what was actually deployed
- ✓ Compliance and audit become difficult
- ✓ Reusing the same tested artifact improves reliability

## ► WHERE JFROG ARTIFACTORY FITS IN CI/CD



## ARTIFACTORY BENEFITS

- ✓ Single source of truth for binaries
- ✓ Versioned and immutable artifacts
- ✓ High availability and scalability
- ✓ Faster builds with caching
- ✓ Security, access control, and audit
- ✓ Works with all major tools

## ► SIMPLE FLOW: CODE → BUILD → ARTIFACT → DEPLOY





## 2. UNDERSTANDING JFROG AND ARTIFACTORY

### WHAT IS JFROG?



JFrog is a company that provides a complete DevOps platform to manage, secure, and distribute software artifacts.

### WHAT DOES ARTIFACTORY DO?



### ARTIFACT REPOSITORY BASICS



An artifact repository is a centralized place where build outputs and packages are stored, versioned, managed, and shared.

### WHAT CAN ARTIFACTORY STORE?



**PACKAGES**  
(MAVEN, NPM, PYPI, ETC.)



**BINARIES**  
(JAR, ZIP, DLL, EXE, ETC.)



**IMAGES**  
(DOCKER IMAGES)



**BUILD OUTPUTS**  
(LOGS, REPORTS, ETC.)

### ARTIFACTORY VS GITHUB

| FEATURE         | ARTIFACTORY                                    | GITHUB                                 |
|-----------------|------------------------------------------------|----------------------------------------|
| PRIMARY PURPOSE | Store and manage binary artifacts and packages | Store and manage source code           |
| WHAT IT STORES  | Build artifacts, binaries, packages, images    | Source code, text files, documentation |
| VERSIONING      | Immutable versions of build outputs            | Code versions with commits             |
| DEPLOYMENT USE  | Used in CI/CD to deploy applications           | Source code is built before deployment |
| ACCESS PATTERNS | Read/Write artifacts in pipelines and builds   | Clone, pull, push code                 |
| EXAMPLES        | Maven, Docker, npm, PyPI, Helm, APT, YUM       | Git repositories                       |

### WHAT SHOULD BE STORED IN ARTIFACTORY

- ✓ Compiled binaries (JAR, ZIP, DLL, EXE)
- ✓ Packages (Maven, npm, PyPI, NuGet, etc.)
- ✓ Container images
- ✓ Helm charts
- ✓ Deb/RPM packages
- ✓ Build outputs and reports
- ✓ Release candidates and production artifacts
- ✓ Artifacts needed for deployment



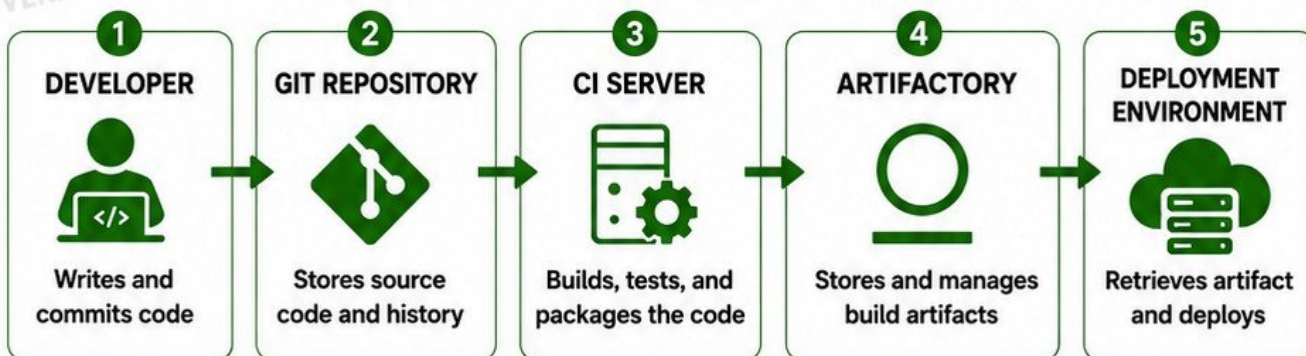
### WHAT SHOULD NOT BE STORED IN ARTIFACTORY

- ✗ Source code (use GitHub, GitLab, Bitbucket)
- ✗ Large datasets or backups
- ✗ Sensitive credentials or plaintext secrets
- ✗ Personal files or documents
- ✗ Temporary local files
- ✗ Anything not part of the build or deploy process
- ✗ Obsolete artifacts (clean up old versions)



# 3. YOUR FIRST ARTIFACTORY ARCHITECTURE

## THE SIMPLE DEVOPS ARTIFACT FLOW



### HOW AN ARTIFACT MOVES THROUGH THE PIPELINE

|   |                                                                                     |                                                                                                                                                                     |
|---|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 |   | <b>DEVELOPER COMMITS CODE</b><br>The developer writes code and pushes it to the Git repository.                                                                     |
| 2 |  | <b>GIT REPOSITORY STORES THE CODE</b><br>The repository keeps all versions of the source code.                                                                      |
| 3 |  | <b>CI SERVER BUILDS AND TESTS</b><br>The CI server pulls the code, builds the application, runs tests, and creates an artifact (JAR, WAR, Docker image, ZIP, etc.). |
| 4 |  | <b>ARTIFACT IS PUBLISHED TO ARTIFACTORY</b><br>The artifact is uploaded to JFrog Artifactory where it is stored, versioned, and available for the team.             |
| 5 |  | <b>ARTIFACT IS RETRIEVED AND DEPLOYED</b><br>The deployment environment downloads the tested artifact from Artifactory and deploys it to staging or production.     |

### SIMPLE ARCHITECTURE DIAGRAM



### WHY THIS ARCHITECTURE MATTERS

|                                                                                     |                                                                  |                                                                                     |                                                              |                                                                                     |                                                                    |                                                                                      |                                                               |                                                                                       |                                                                  |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------|
|  | <b>CONSISTENCY</b><br>Everyone deploys the same tested artifact. |  | <b>RELIABILITY</b><br>Avoids "works on my machine" problems. |  | <b>REUSABILITY</b><br>Artifacts can be reused across environments. |  | <b>SECURITY</b><br>Controlled access and immutable artifacts. |  | <b>TRACEABILITY</b><br>Know exactly what was built and deployed. |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------|



# 4. INSTALLING AND ACCESSING ARTIFACTORY

## JFROG CLOUD VS SELF-HOSTED

### JFROG CLOUD

- ✓ Managed by JFrog
- ✓ Quick sign up
- ✓ No infrastructure to manage
- ✓ Automatic updates
- ✓ High availability
- ✓ Pay as you go

### SELF-HOSTED

- ✓ Deployed on your infrastructure
- ✓ Full control
- ✓ Customize as needed
- ✓ You handle backups, updates, scaling
- ✓ Best for on-prem requirements

## BASIC INSTALLATION CONCEPTS

- 1 Prerequisites (Java, memory, disk space)
- 2 Download Artifactory (OSS, Pro, or Enterprise)
- 3 Extract and configure
- 4 Start Artifactory service
- 5 Verify Artifactory is running
- 6 Access using the platform URL



**TIP**

For beginners, JFrog Cloud is the fastest way to get started.

## WEB INTERFACE



## FIRST LOGIN

- 1 Open your platform URL in a browser
- 2 Enter default username (admin)
- 3 Enter the initial password from setup
- 4 Change the password immediately
- 5 Start exploring the platform

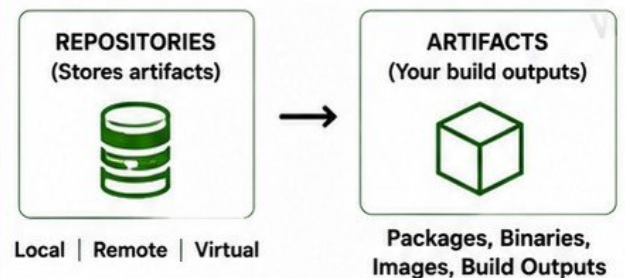
## PLATFORM URL EXAMPLES

|                                                                                                 |                                                                         |
|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
|  JFROG CLOUD | <a href="https://your-domain.jfrog.io">https://your-domain.jfrog.io</a> |
|  SELF-HOSTED | <a href="http://your-server:8081">http://your-server:8081</a>           |

## BASIC NAVIGATION

|                                                                                     |                                                                 |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------|
|  | <b>HOME</b> – Overview of system status and recent activity     |
|  | <b>ARTIFACTS</b> – Browse, search, and manage artifacts         |
|  | <b>REPOSITORIES</b> – Create and manage repositories            |
|  | <b>SEARCH</b> – Powerful search across artifacts and properties |
|  | <b>BUILDS</b> – View build information and build info retention |
|  | <b>SECURITY</b> – Users, groups, permissions, tokens            |
|  | <b>TOOLS</b> – JFrog CLI, REST API, and integrations            |
|  | <b>ADMIN</b> – System configuration and maintenance             |

## WHERE REPOSITORIES AND ARTIFACTS LIVE



VERIQTA SOCIAL



/ veriqta



# 5. REPOSITORY TYPES

ARTIFACTORY HAS 3 TYPES OF REPOSITORIES

## 1. LOCAL REPOSITORY



Stores artifacts directly inside Artifactory.  
You deploy/push artifacts into it.

### USES

- ✓ Store your own build outputs
- ✓ Internal binaries and packages
- ✓ Releases and snapshots

### EXAMPLE

Store your application JAR, Docker images, ZIP files

## 2. REMOTE REPOSITORY



Connects to an external repository (like Maven Central, npm, Docker Hub) and caches artifacts.

### USES

- ✓ Proxy external repositories
- ✓ Improve build speed (caching)
- ✓ Work offline after first download
- ✓ Reduce external dependency risks

### EXAMPLE

Maven Central, npmjs.org, Docker Hub

## 3. VIRTUAL REPOSITORY



Combines multiple local and/or remote repositories behind a single URL.

### USES

- ✓ Provide one URL for developers
- ✓ Search across multiple repos
- ✓ Simplify dependency resolution
- ✓ Control repository priority

### EXAMPLE

Combine local releases + remote Maven Central

## WHY ALL THREE EXIST?



### PERFORMANCE

Remote repos cache external artifacts. Local repos store your own builds.



### RELIABILITY

Builds don't break when external repositories are down.



### CONTROL

You decide what gets stored, what is allowed, and how it is accessed.



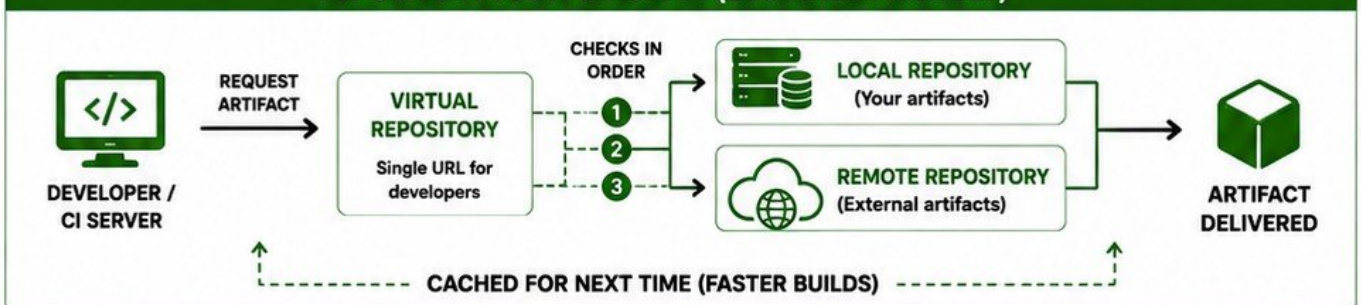
### SIMPLICITY

Virtual repos give developers one clean and easy URL to use.

## WHEN TO USE EACH?

| REPOSITORY TYPE                                                                             | WHEN TO USE                                                       | EXAMPLE SCENARIO                                                               |
|---------------------------------------------------------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------------------------|
|  LOCAL   | When you want to store your own artifacts and build outputs.      | Your CI pipeline publishes the application JAR to a local repository.          |
|  REMOTE  | When you need artifacts from external sources and want caching.   | Developers download dependencies from Maven Central through Artifactory.       |
|  VIRTUAL | When you want a single point of access for multiple repositories. | Developers use one URL that provides both internal releases and Maven Central. |

## REPOSITORY FLOW DIAGRAM (BEGINNER FRIENDLY)







# 6. CREATING YOUR FIRST LOCAL REPOSITORY

## STEP-BY-STEP: STORE YOUR FIRST ARTIFACT IN ARTIFACTORY

**1**

**CREATE A REPOSITORY**

Go to Admin > Repositories Click "New Repository"

**2**

**CHOOSE PACKAGE TYPE**

Select the package type you want to store.

- ☒ Maven
- ☐ npm
- ☐ Docker
- ☐ Generic
- ☐ PyPI
- ☐ Helm
- ☐ NuGet

**3**

**REPOSITORY KEY**

Enter a unique repository key. This becomes part of the repository URL.

Repository Key \*

Repository Layout

Simple Default

Create

**4**

**UPLOAD AN ARTIFACT**

Go to your new repository and click "Deploy" to upload your artifact.

Deploy

or use CLI:

```
jfrog rt u file.jar my-local-repo/
```

**5**

**BROWSE REPOSITORY CONTENTS**

Your artifact is now stored in the repository. Browse the files and folders.

**6**

**DOWNLOAD THE ARTIFACT**

Click the file and choose "Download" the artifacts.

Download

or use CLI:

```
jfrog rt dl my-local-repo/com/example/app/1.0.0/app-1.0.0.jar
```

**7**

**DOWNLOAD THE ARTIFACT**

Click the file and choose "Download" or use CLI.

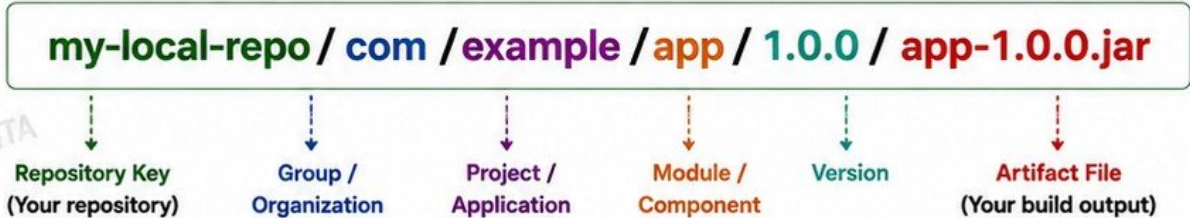
Download

or use CLI:

```
jfrog rt dl my-local-repo/com/example/app/1.0.0/app-1.0.0.jar
```

### 7 UNDERSTAND THE ARTIFACT PATH

Every artifact has a path that shows where it lives in the repository.



#### SUMMARY

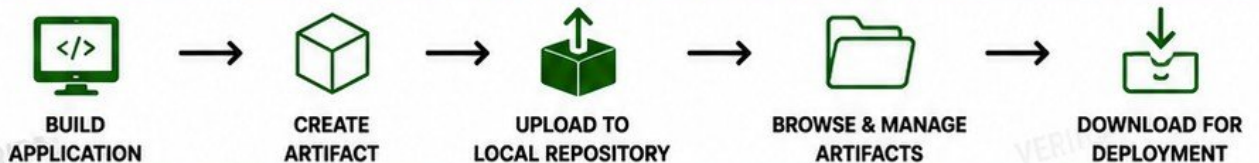
- ✓ You created a local repository.
- ✓ You downloaded the artifact.
- ✓ You uploaded an artifact.
- ✓ You understood the artifact path structure.
- ✓ You browsed and verified its location.



#### TIP

Local repositories are best for storing your own build artifacts and releases.

#### SIMPLE FLOW: STORE AND RETRIEVE AN ARTIFACT



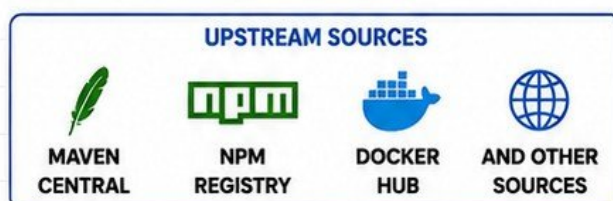
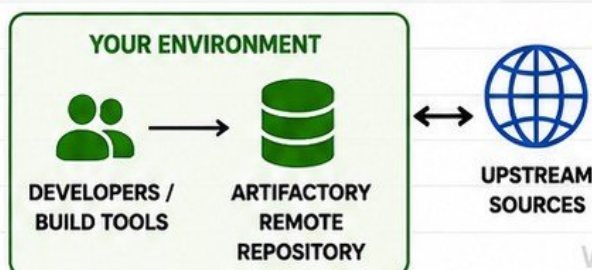


# REMOTE REPOSITORIES

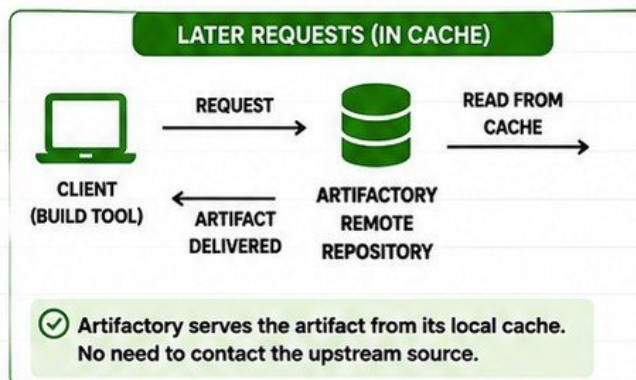
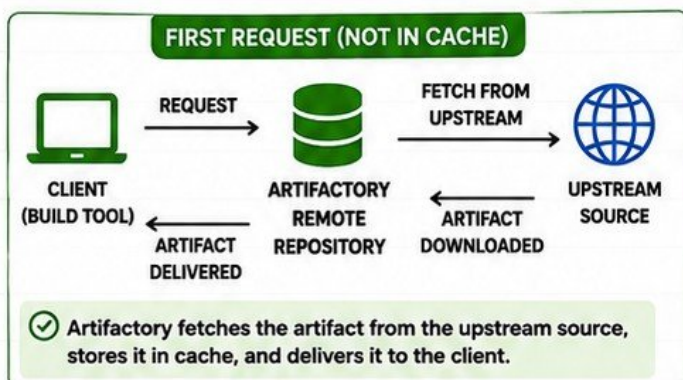


## WHY PROXY EXTERNAL REPOSITORIES

- Protects your builds from external outages.
- Improves build speed through local caching.
- Reduces bandwidth usage and external dependency.
- Provides consistent and repeatable builds.
- Enforces security and controls what can be downloaded.



## HOW REMOTE REPOSITORIES WORK



## WHY REMOTE REPOSITORIES IMPROVE RELIABILITY

- Builds do not fail if the external source is down.
- Faster builds due to local caching.
- Lower network usage and faster downloads.
- Consistent versions stay cached for repeatable builds.
- Central point to control and audit external dependencies.



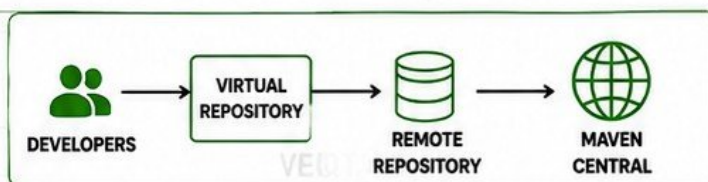
## BEST PRACTICES

- Always use remote repositories for external sources.
- Use virtual repositories for clients.
- Set proper cache retention policies.
- Use checksum validation for security.
- Limit direct access to upstream sources.



## EXAMPLE USE CASE

Your team uses Maven and depends on libraries from Maven Central. Instead of every developer machine connecting to Maven Central, all requests go to Artifactory remote repository first. Artifactory caches the libraries and serves them quickly and reliably.





# VIRTUAL REPOSITORIES

## COMBINING MULTIPLE REPOSITORIES BEHIND ONE ENDPOINT

### WHAT IS A VIRTUAL REPOSITORY?



A virtual repository combines multiple repositories (local and remote) and exposes them behind a single endpoint.

Clients only need **ONE URL** to resolve all dependencies.

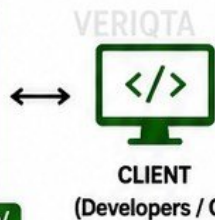
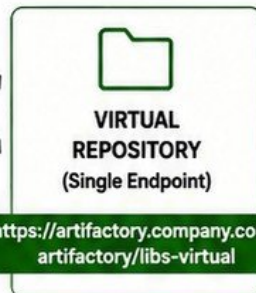
### LOCAL + REMOTE REPOSITORY PATTERN



LOCAL REPOSITORY  
(Your Artifacts)



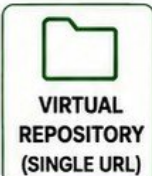
REMOTE REPOSITORY  
(Upstream Source)



### HOW CLIENTS USE VIRTUAL REPOSITORIES



1. REQUEST  
DEPENDENCY



2. RETURNS  
ARTIFACT



DEPENDENCY  
RESOLVED



Clients do not need to know or care if the artifact comes from a local repository or a remote repository.

### REPOSITORY RESOLUTION ORDER

Artifactory checks the repositories in the virtual repository in a specific order until it finds the artifact.

- 1 Check Local Repository (Your Artifacts) (faster, preferred)
- 2 If not found, check Remote Repository (Upstream Source)
- 3 If found in Remote, it is downloaded and cached (for future requests)
- 4 Return artifact to the client

RESOLUTION  
ORDER



### WHY DEVELOPERS SHOULD NOT MANAGE MANY REPOSITORY URLS



SIMPLIFIES  
CONFIGURATION

One URL for everything.  
Easier setup for all tools.



REDUCES  
ERRORS

Fewer URLs mean fewer  
misconfigurations.



IMPROVES  
PERFORMANCE

Local repos are checked  
first (faster resolution).



ENHANCES  
SECURITY

Centralized control over  
what is available.



BETTER  
EXPERIENCE

Developers focus on code,  
not repository management.

### EXAMPLE: MAVEN VIRTUAL REPOSITORY

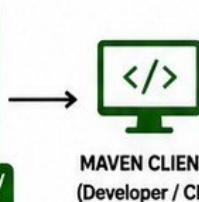
#### BACKEND REPOSITORIES



LOCAL MAVEN  
REPOSITORY  
Company releases  
and snapshots



REMOTE MAVEN  
(REMOTE)  
Maven Central and  
other public repos



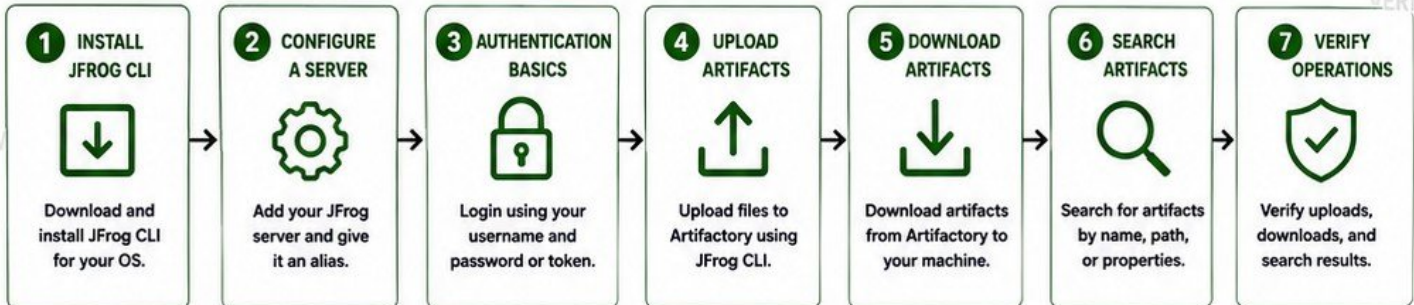
#### FLOW

- 1 Client requests a dependency via the virtual repo.
- 2 Artifactory checks local repo first.
- 3 If not found, checks remote repo.
- 4 If found in remote, caches it.
- 5 Returns artifact to the client.



# WORKING WITH JFROG CLI

## ESSENTIAL COMMANDS FOR EVERY BEGINNER



### 1 INSTALL JFROG CLI



```
# Windows (PowerShell)
scoop install jfrog-cli

# macOS (Homebrew)
brew install jfrog-cli

# Linux (curl)
curl -fL https://getcli.jfrog.io | sh
```

### 2 CONFIGURE A SERVER

Add your Artifactory server with an alias.

```
jfrog rt c add my-artifactory \
--url=https://artifactory.company.com \
--user=admin --password=*****
```

✓ This creates a secure configuration stored locally.

### 3 AUTHENTICATION BASICS

Login to the configured server.

```
jfrog rt login my-artifactory
```

✓ You can also use Access Tokens:

```
jfrog rt c add my-artifactory \
--access-token=YOUR_TOKEN
```

### 4 UPLOAD ARTIFACTS

Upload a file to a repository.

```
jfrog rt u "app.jar" my-repo/ \
--build-name=my-build \
--build-number=1.0.0
```

✓ Use --recursive to upload a whole folder.

```
jfrog rt u "dist/" my-repo/ --recursive
```

### 5 DOWNLOAD ARTIFACTS

Download a file from a repository.

```
jfrog rt d "my-repo/app.jar"
./downloads/
```

✓ Use --recursive to download entire folders.

```
jfrog rt dl "my-repo/*" ./downloads/
```

### 6 SEARCH ARTIFACTS

Search by name, path, or properties.

```
jfrog rt s "libs-release-local/"
jfrog rt s "name=app*",
"@build.name=my-build" --spec
```

✓ Use --spec for advanced search (properties, date, size, etc.).

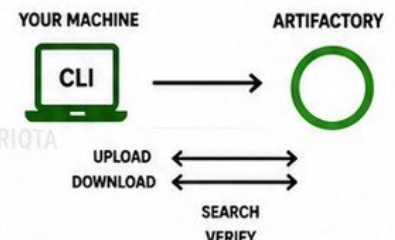
### 7 VERIFY SUCCESSFUL OPERATIONS

Check uploaded files, builds, and server connection.

|                                          |                                |
|------------------------------------------|--------------------------------|
| <code>jfrog rt ping</code>               | Test connection to the server. |
| <code>jfrog rt bp</code>                 | List all builds.               |
| <code>jfrog rt bce my-build 1.0.0</code> | Show build details.            |
| <code>jfrog rt ls my-repo/</code>        | List files in a repository.    |
| <code>jfrog rt sp</code>                 | Show storage summary.          |

### EXAMPLE WORKFLOW

- 1 Install JFrog CLI
- 2 Configure a server
- 3 Login to the server
- 4 Upload an artifact
- 5 Download the artifact
- 6 Search for the artifact
- 7 Verify the operation



#### TIP

Always reference the virtual repository URL in your build tools, not the local or remote repositories directly. This makes your builds simpler, faster, and more reliable.



# MAVEN AND ARTIFACTORY

## 1. MAVEN PACKAGE LIFECYCLE

### Standard Maven Build Lifecycle



- Package creates the artifact (JAR/WAR). Install adds it to the local repo. Deploy publishes to remote repo.

## 2. MAVEN REPOSITORIES



## 3. MAVEN SETTINGS.XML

Configure Maven to use Artifactory.

Sample settings.xml

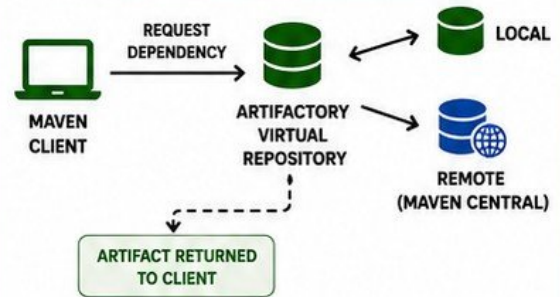
```

<settings>
  <servers>
    <server>
      <id>my-artifactory</id>
      <username>admin</username>
      <password>*****</password>
    </server>
  </servers>
  <mirrors>
    <mirror>
      <id>artifactory-mirror</id>
      <mirrorOf>*</mirrorOf>
      <url>https://artifactory.company.com/artifactory/libs-virtual</url>
    </mirror>
  </mirrors>
</settings>
  
```

What this does:

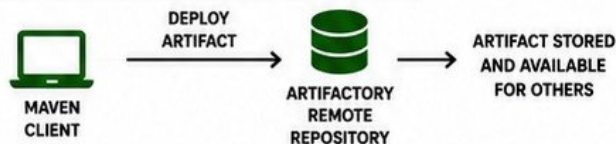
- Authenticates with Artifactory.
- Uses Virtual Repository for all dependencies.
- No need to contact Maven Central directly.

## 4. RESOLVING DEPENDENCIES THROUGH ARTIFACTORY



Artifactory checks local first. If not found, it fetches from remote and caches it.

## 5. PUBLISHING BUILD ARTIFACTS



- Use the deploy phase to publish your artifact.

```
mvn clean deploy
```

## 6. SNAPSHOT VS RELEASE ARTIFACTS

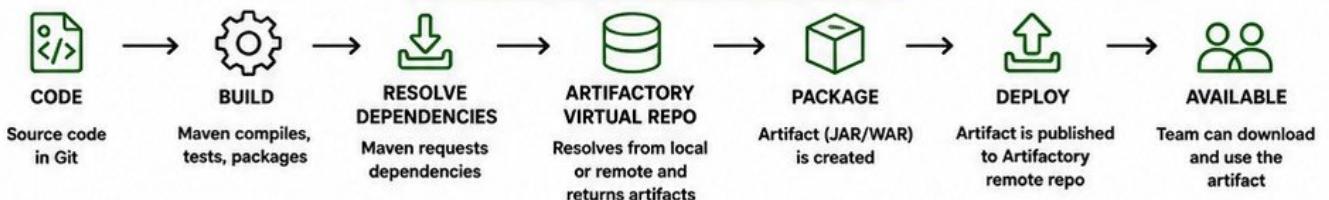
### SNAPSHOT ARTIFACTS

- Used during development.
- Mutable – can be overwritten.
- Always ends with -SNAPSHOT
- Example: 1.0.0-SNAPSHOT
- Stored in snapshot repositories.

### RELEASE ARTIFACTS

- Used in production.
- Immutable – cannot be overwritten.
- No -SNAPSHOT suffix.
- Example: 1.0.0
- Stored in release repositories.

## 7. MAVEN → ARTIFACTORY FLOW



### KEY BENEFITS

- Faster builds (due to caching)
- Consistent and reliable dependencies
- Centralized artifact management
- Secure and controlled access

### COMMON REPOSITORY LAYOUT

libs-release-local (Your released artifacts)  
 libs-snapshot-local (Your snapshot artifacts)  
 libs-remote (Proxy for external repos)  
 libs-virtual (Virtual repo for clients)

### BEST PRACTICE

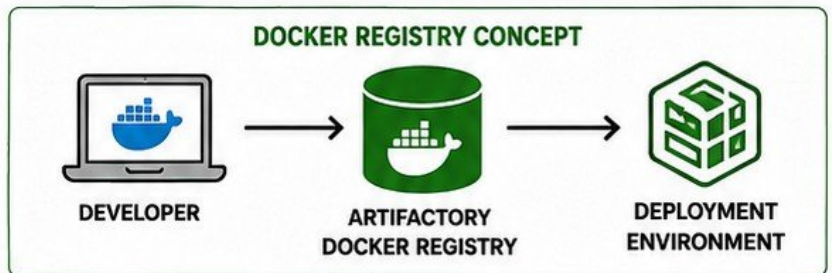
- Always use a Virtual Repository in Maven settings.xml. Do not connect directly to Maven Central.



# DOCKER IMAGES AND ARTIFACTORY

## 1 DOCKER REGISTRY BASICS

- A Docker registry stores and distributes Docker images.
- Images contain everything needed to run a container.
- Registries allow versioning, access control, and secure distribution.
- Artifactory can act as your private Docker registry.

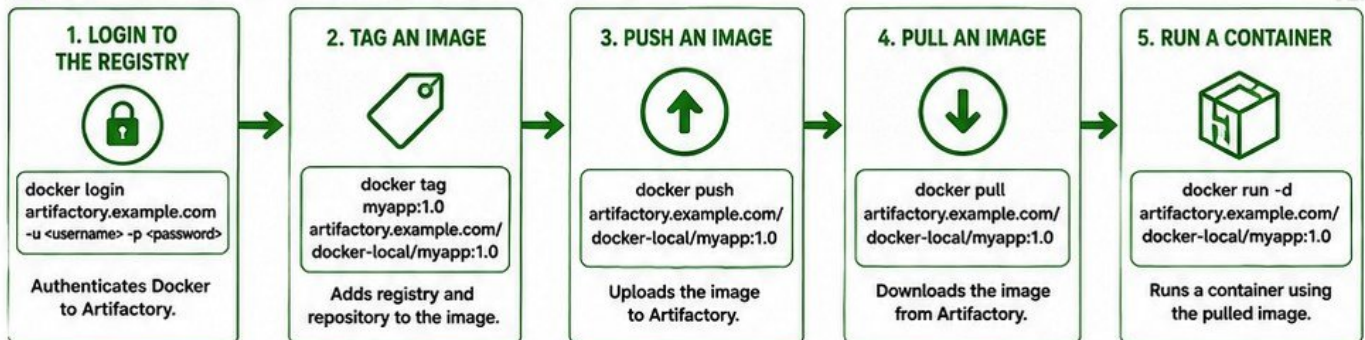


## 2 DOCKER REPOSITORIES IN ARTIFACTORY

- Create Docker (local) repositories in Artifactory.
- Repository format: docker
- Store private Docker images safely.
- Apply permissions and security policies.
- Organize images by team or application.

| EXAMPLE DOCKER REPOSITORIES |         |                               |
|-----------------------------|---------|-------------------------------|
| REPOSITORY KEY              | TYPE    | DESCRIPTION                   |
| docker-local                | Local   | Stores custom Docker images   |
| docker-release              | Local   | Stores release images         |
| docker-cache                | Remote  | Proxy to Docker Hub           |
| docker-virtual              | Virtual | Combines local + remote repos |

## 3 WORKING WITH DOCKER IMAGES IN ARTIFACTORY



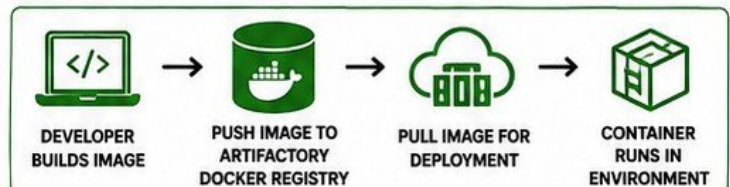
## 4 IMAGE TAGS AND VERSIONS

- Tags identify image versions.
- Common tags: latest, v1.0, v1.0.1, prod, dev.
- Avoid using latest in production.
- Use meaningful and immutable tags.
- Example: myapp:1.0.0 instead of latest

### TAG EXAMPLES

|                  |   |
|------------------|---|
| myapp:latest     | ✗ |
| myapp:1.0        | ✓ |
| myapp:1.0.1      | ✓ |
| myapp:prod       | ✓ |
| myapp:2024-05-01 | ✓ |

## 5 DOCKER → ARTIFACTORY → DEPLOYMENT FLOW



### BENEFITS OF USING ARTIFACTORY AS DOCKER REGISTRY



### BEST PRACTICES

- ✓ Always tag images with a specific version.
- ✓ Do not use "latest" in production.
- ✓ Scan images for vulnerabilities.
- ✓ Use repositories for different environments.
- ✓ Keep images small and secure.





VERIQTA

VERIQTA

VERIQTA

VERIQTA



# AND ARTIFACTORY

VERIQTA

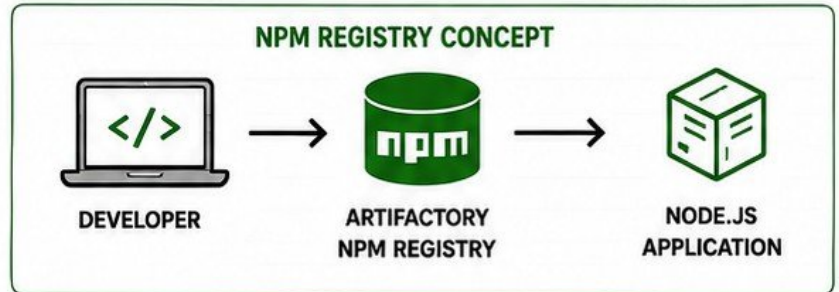
VERIQTA

BUILD. STORE. SHARE. DEPLOY.

VERIQTA

## 1 NPM REGISTRY BASICS

- The npm registry stores and distributes JavaScript packages.
- Packages can be public or private.
- Artifactory acts as your private npm registry.
- It helps you control, secure, and cache dependencies.
- Reduces dependency on the public internet and improves reliability.



## 2 CONFIGURE NPM TO USE ARTIFACTORY

Update your .npmrc file or run the command below.

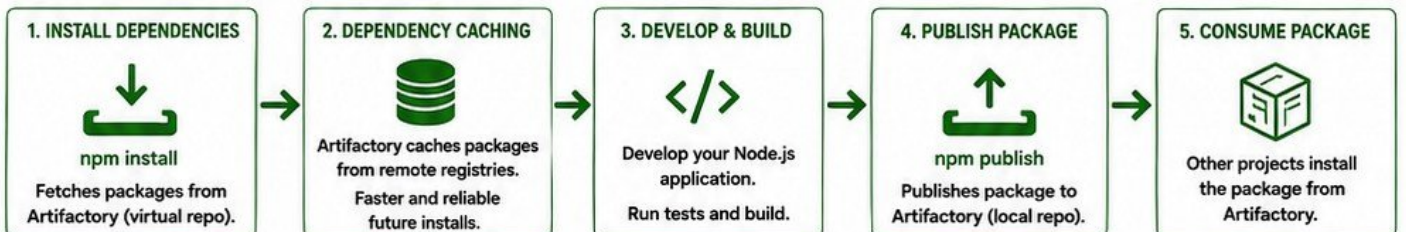
```
registry=https://artifactory.example.com/artifactory/api/npm/npm/
//artifactory.example.com/artifactory/api/npm/npm/:_authToken=YOUR_TOKEN
always-auth=true
```

- ✓ This tells npm to use Artifactory as the registry for install and publish operations.

## 3 NPM REPOSITORIES IN ARTIFACTORY

| REPOSITORY TYPE | DESCRIPTION                               | USE CASE                            |
|-----------------|-------------------------------------------|-------------------------------------|
| LOCAL           | Stores your private npm packages.         | Publish internal packages.          |
| REMOTE          | Proxies public registries like npmjs.org. | Cache external packages.            |
| VIRTUAL         | Combines local and remote repositories.   | Used by developers and build tools. |

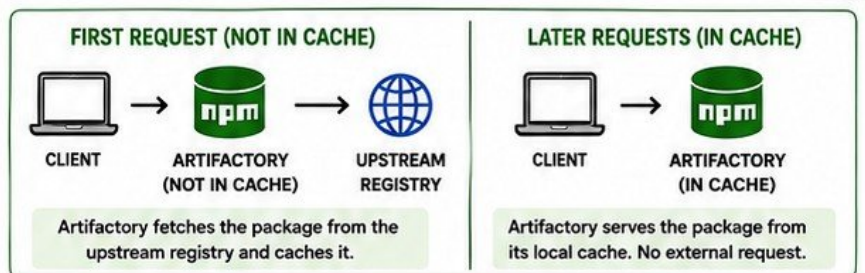
## 4 NPM WORKFLOW WITH ARTIFACTORY



## 5 DEPENDENCY CACHING



- First request for a package → Artifactory fetches it from the upstream registry and stores it.
- Next requests → Served from Artifactory cache.
- Greatly improves speed and reduces internet dependency.



## 6 PUBLISHING PACKAGES

- Create or build your npm package.
- Version your package in package.json.
- Run npm publish.
- Package is uploaded to your Artifactory local repository.
- Other teams can install it securely.

### PACKAGE VERSION EXAMPLE

```
{
  "name": "my-lib",
  "version": "1.0.0",
  "description": "My Library",
  "main": "index.js"
}
```



## 7 SIMPLE NODE.JS WORKFLOW



### BENEFITS OF USING ARTIFACTORY FOR NPM



SECURE PRIVATE PACKAGES



FASTER INSTALLS WITH LOCAL CACHING



CONTROL ACCESS AND PERMISSIONS



RELIABLE BUILDS AND REPRODUCIBLE ENVIRONMENTS



MINIMIZE DEPENDENCY ON PUBLIC REGISTRIES

VERIQTA SOCIAL



/ veriqta

VERIQTA

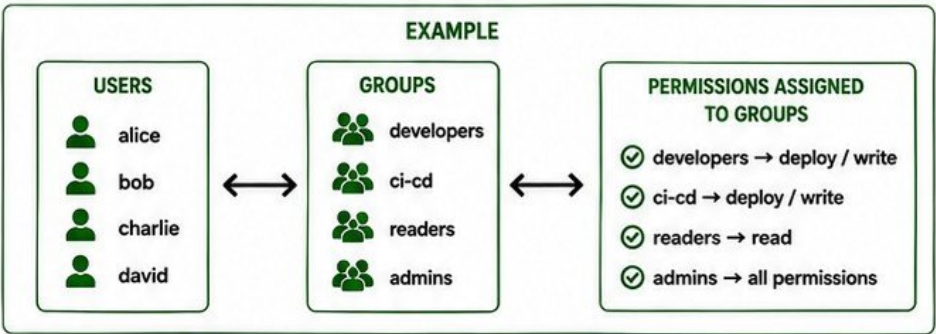


# USERS, GROUPS, PERMISSIONS, AND ACCESS

## CONTROL WHO CAN ACCESS WHAT

### 1 USERS AND GROUPS

- Users are individuals who access Artifactory.
- Groups are collections of users.
- Permissions are assigned to groups, not individual users.
- This makes management simple and scalable.

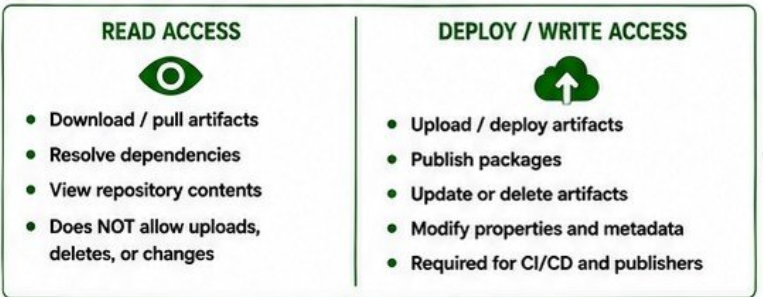


### 2 PERMISSION TARGETS

Permissions are applied to specific targets:

- REPOSITORIES**  
Control access to specific repositories.
- REPOSITORY PATHS**  
Apply permissions to specific folders/paths.
- USERS / GROUPS**  
Manage who gets access.
- TOKENS / API KEYS**  
Control access for automation and integrations.
- GLOBAL**  
System-level permissions (use with caution).

### 3 READ VS DEPLOY/WRITE ACCESS



**FOLLOW THE PRINCIPLE OF LEAST PRIVILEGE**  
Give only the access needed to perform the job.

### 4 REPOSITORY PERMISSIONS EXAMPLE

| GROUP      | REPOSITORY         | PERMISSION     | ACCESS LEVEL                |
|------------|--------------------|----------------|-----------------------------|
| developers | libs-release-local | deploy / write | Upload and download         |
| ci-cd      | docker-local       | deploy / write | Push and pull Docker images |
| readers    | libs-release-local | read           | Download only               |
| guests     | public-repo        | read           | Download public artifacts   |

### 5 SERVICE ACCOUNTS

- Service accounts are non-human users.
- Used by CI/CD tools, automation, and scripts.
- Always use access tokens instead of passwords.
- Give only the permissions required.
- Example: "jenkins-ci" user with deploy access to specific repositories.



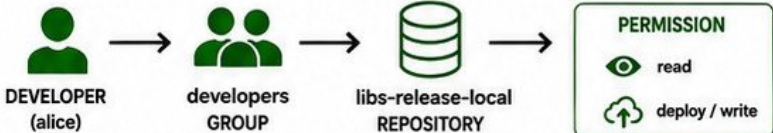
### 6 WHY DEVELOPERS SHOULD NOT GET ADMIN ACCESS

- ❗ Admins can delete or overwrite critical artifacts.
- ❗ Admins can change permissions and break security.
- ❗ Admins can access all repositories and data.
- ❗ Increases risk of accidental or malicious actions.



**ADMIN ACCESS IS ONLY FOR TRUSTED ADMINS AND AUTOMATION, NOT DAILY USERS.**

### 7 LEAST-PRIVILEGE EXAMPLE



Alice can upload and download to libs-release-local, but cannot delete, change permissions, or access other repositories.

### 8 BEST PRACTICES



USE GROUPS, NOT INDIVIDUAL PERMISSIONS.



GIVE MINIMUM ACCESS NEEDED TO GET THE JOB DONE.



USE TOKENS FOR AUTOMATION AND SERVICE ACCOUNTS.



REVIEW PERMISSIONS REGULARLY.



MONITOR ACCESS AND ACTIVITY LOGS.



# AUTHENTICATION AND TOKENS

## SECURE ACCESS TO ARTIFACTORY

### 1 PASSWORDS VS ACCESS TOKENS

| PASSWORDS                                                                                                                                                                                                                            | ACCESS TOKENS                                                                                                                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>❌ Tied to a specific user</li><li>❌ Harder to manage and rotate</li><li>❌ Risky for automation and CI/CD</li><li>❌ May expire with user changes</li><li>❌ Broader access than needed</li></ul> | <ul style="list-style-type: none"><li>✅ Not tied to a user</li><li>✅ Easy to manage and rotate</li><li>✅ Ideal for automation and CI/CD</li><li>✅ Can have expiration date</li><li>✅ Minimum permissions possible</li></ul> |
| <div> <b>USE TOKENS FOR AUTOMATION.</b><br/><b>USE PASSWORDS ONLY FOR HUMAN INTERACTION.</b></div>                                                  |                                                                                                                                                                                                                             |

### 2 JFROG ACCESS TOKENS

- Created in Artifactory under your user.
- Can be scoped to specific permissions.
- Can have an expiration date.
- Can be revoked at any time.
- Used by CI/CD tools, scripts, and integrations.

#### TOKEN EXAMPLE



eyJ2ZXliOi...K3d9f8a2b7c6d4e



Expires: 2025-07-20  
Permissions: deploy / read

### 3 CI/CD AUTHENTICATION



### 4 TOKEN EXPIRATION



- Set an expiration date for tokens.
- Prevents long-term exposure.
- Rotate tokens regularly.
- Monitor token usage.



**A SHORT-LIVED TOKEN IS  
A SAFER TOKEN.**

### 5 AVOID HARD-CODED CREDENTIALS

#### DON'T DO THIS

```
export JFROG_USER="admin"  
export JFROG_PASSWORD="mypassword"
```



**HARD-CODED CREDENTIALS CAN LEAK AND BE ABUSED.  
ALWAYS USE TOKENS AND SECURE STORAGE.**

#### DO THIS INSTEAD

```
export JFROG_URL="https://artifactory.example.com"  
export JFROG_ACCESS_TOKEN="<your_token>"
```

### 6 SECRETS MANAGEMENT



- Store tokens in a secrets manager. (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault, etc.)
- Do not store tokens in code, scripts, or environment files.
- Limit access to secrets.
- Audit secret access regularly.

### 7 BASIC CREDENTIAL SECURITY



Use minimum permissions required for the job.



Rotate tokens regularly.



Monitor access and usage.



Revoke unused or old tokens.



**SECURE TODAY,  
SAFE TOMORROW.**

### 8 BEST PRACTICES

- ✅ Use access tokens for automation.
- ✅ Give tokens the least privileges possible.
- ✅ Set token expiration and rotate regularly.
- ✅ Store tokens securely in a secrets manager.
- ✅ Never commit tokens to repositories.
- ✅ Review and revoke unused tokens.
- ✅ Audit authentication logs regularly.



**STRONG AUTHENTICATION  
PROTECTS YOUR  
ARTIFACTS AND SYSTEMS.**



#### KEY TAKEAWAY

**Tokens + Least Privilege + Secrets Management  
= Secure and Reliable Automation**



# ARTIFACT METADATA AND CHECKSUMS

KNOW EXACTLY WHAT YOU BUILD AND DEPLOY

VERIQTA

## 1 ARTIFACT PROPERTIES (METADATA)



- Key-value information attached to artifacts.
- Helps you categorize, search, filter, and govern artifacts.
- Examples:

| PROPERTY KEY  | VALUE EXAMPLE   |
|---------------|-----------------|
| project       | payment-service |
| environment   | production      |
| team          | platform        |
| version       | 1.2.3           |
| build.number  | 105             |
| branch        | main            |
| commit        | a1b2c3d4        |
| created.by    | jenkins         |
| os            | linux           |
| artifact.type | jar             |

## 2 BUILD INFORMATION



- Captures details about how an artifact was built.
- Links artifacts to the build that created them.
- Includes:

|  |                                          |
|--|------------------------------------------|
|  | <b>BUILD NAME</b><br>payment-service     |
|  | <b>BUILD NUMBER</b><br>105               |
|  | <b>BUILD TIME</b><br>2025-07-20 10:15:30 |
|  | <b>BUILT BY</b><br>jenkins               |
|  | <b>SCM REVISION</b><br>a1b2c3d4          |
|  | <b>BRANCH</b><br>main                    |

## 3 CHECKSUMS



- Checksums verify the integrity of an artifact.
- Ensures the file has not been tampered with or corrupted.
- Common algorithms:

| ALGORITHM | PURPOSE                                     |
|-----------|---------------------------------------------|
| SHA-256   | Strong integrity verification (recommended) |
| SHA-1     | Legacy use (weaker)                         |
| MD5       | Not recommended (weak)                      |

### EXAMPLE SHA-256 CHECKSUM

```
a3f5d8e9b7c4d3e2f1a0b9c8d7e6f5a4
3b2c1d0e9f8a7b6c5d4e3f2a1b0c9d8
```

## 4 SHA-256 CONCEPTS

**SHA-256**

INPUT FILE

UNIQUE  
FINGERPRINT  
(HASH)

- Takes any file and generates a 256-bit (64 character) hash.
- Even a tiny change in the file produces a completely different hash.
- Used to verify the file is exactly the same.

## 5 WHY INTEGRITY MATTERS



- Protects against data corruption during transfer.
- Detects tampering or unauthorized changes.
- Ensures you deploy exactly what was built and tested.
- Critical for security, compliance, and audit trails.

**INTEGRITY = TRUST  
DEPLOY WITH CONFIDENCE**

## 6 SEARCHING WITH METADATA



- Search and filter artifacts using properties.
- Find artifacts by project, version, environment, build, branch, or any key.
- Saves time and improves traceability.

### EXAMPLE SEARCH QUERIES

```
project:payment-service AND version:1.2.3
environment:production AND build.number:105
branch:main AND team:platform
```

## 7 IDENTIFYING EXACTLY WHAT WAS BUILT AND DEPLOYED



CODE



BUILD

ARTIFACT  
(JAR / IMAGE / PKG)ARTIFACT  
METADATACHECKSUM  
(SHA-256)

DEPLOYMENT

- ✓ Track every artifact from build to deployment.
- ✓ Verify integrity using checksums.
- ✓ Use metadata to search and audit.
- ✓ Prove exactly what is running in production.

### EXAMPLE: ARTIFACT DETAILS IN ARTIFACTORY

|                    |                                                                 |
|--------------------|-----------------------------------------------------------------|
| ARTIFACT PATH      | libs-release-local/com/example/app/1.2.3/app-1.2.3.jar          |
| SIZE               | 15.4 MB                                                         |
| CHECKSUM (SHA-256) | a3f5d8e9b7c4d3e2f1a0b9c8d7e6f5a43b2c1d0e9f8a7b6c5d4e3f2a1b0c9d8 |
| CREATED            | 2025-07-20 10:15:30                                             |
| BUILD INFO         | payment-service / 105                                           |
| PROPERTIES         | env=production; team=platform; version=1.2.3; branch=main       |

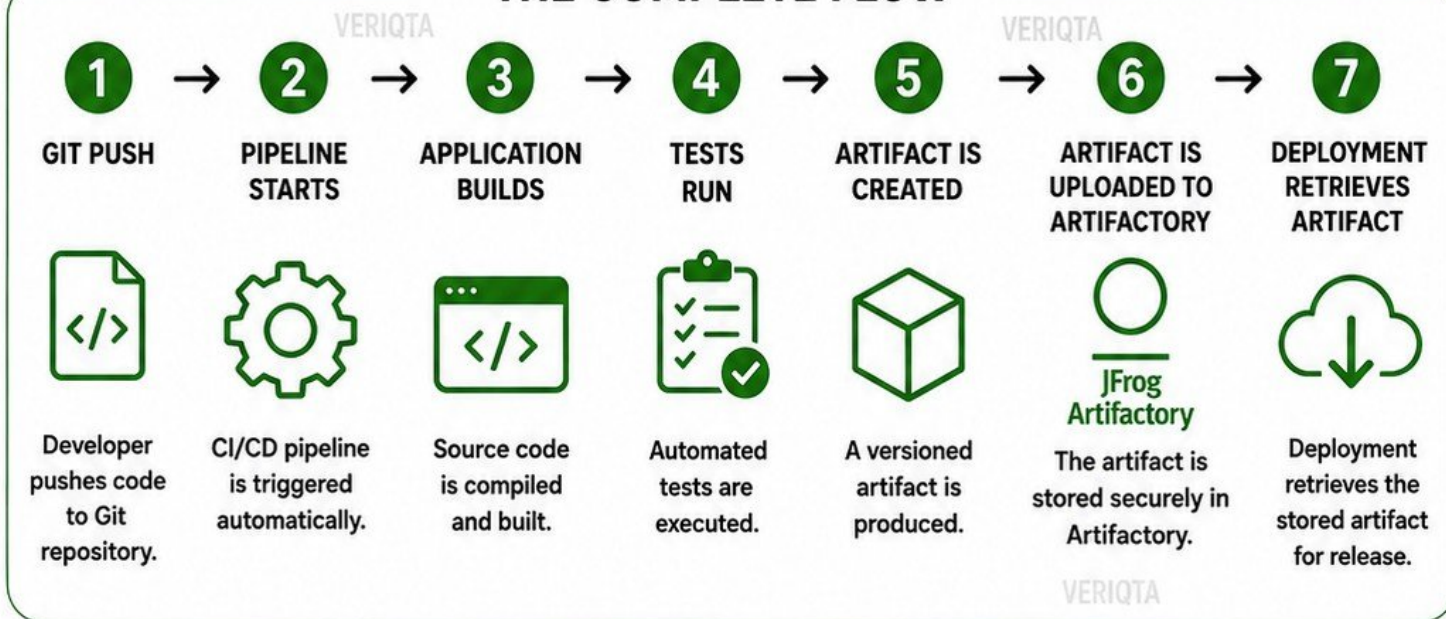
### BENEFITS

- ✓ Full traceability
- ✓ Faster troubleshooting
- ✓ Stronger security
- ✓ Reliable deployments
- ✓ Compliance and audit ready



# ARTIFACTORY IN CI/CD

## THE COMPLETE FLOW



## WHY PRODUCTION SHOULD DEPLOY THE TESTED ARTIFACT



### KEY TAKEAWAY



**BUILD ONCE.  
TEST ONCE.  
DEPLOY MANY TIMES.**

**ARTIFACTORY IS THE SINGLE  
SOURCE OF TRUST.**

### BEST PRACTICES

- ✓ Always deploy the same artifact that passed all tests.
- ✓ Never rebuild in production.
- ✓ Store immutable, versioned artifacts.
- ✓ Use checksums to verify artifact integrity.
- ✓ Control access and permissions to your artifacts.
- ✓ Keep retention policies to manage old artifacts.

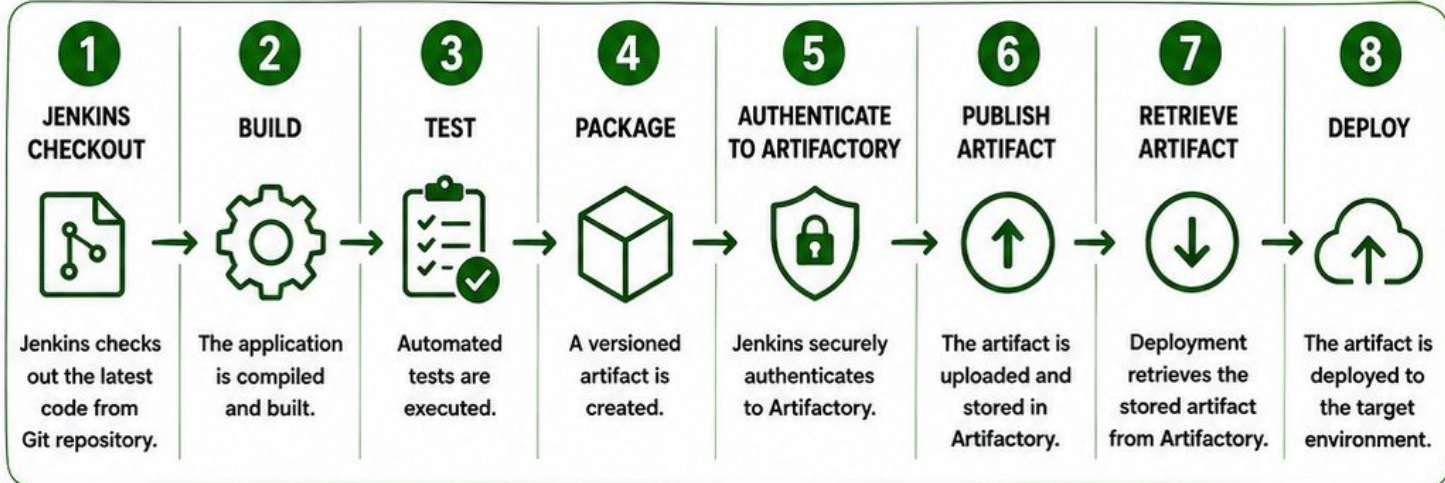


# JENKINS + ARTIFACTORY

## BEGINNER PIPELINE

### PIPELINE STAGES

VERIQTA

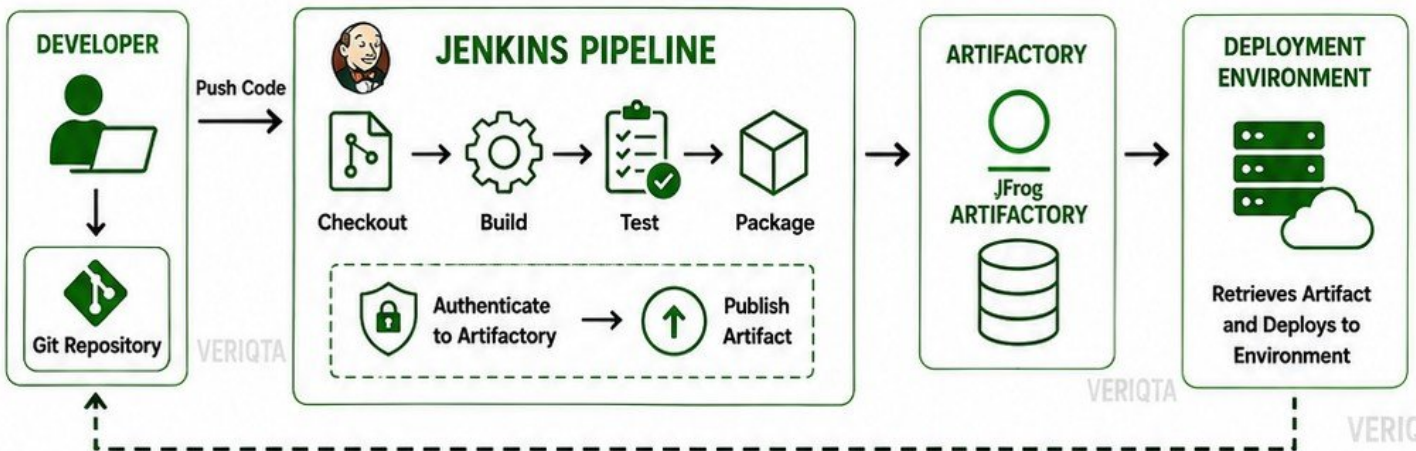


VERIQTA

### SIMPLE PIPELINE ARCHITECTURE

VERIQTA

VERIQTA



#### WHY THIS PIPELINE MATTERS

- Ensures consistency across all environments.
- The exact tested artifact is deployed to production.
- Eliminates "works on my machine" issues.
- Improves security and traceability.
- Faster, reliable, and repeatable deployments.



#### BEST PRACTICES

- Always deploy the same artifact that passed all tests.
- Use versioned artifacts (never overwrite releases).
- Keep build and release processes separate.
- Store artifacts in the right repository.
- Use credentials securely (never hard-code).
- Keep your Artifactory clean with retention policies.



#### KEY TAKEAWAY

Build once, test once, store once, and deploy many times.  
Artifactory is the single source of truth for your artifacts.

VERIQTA

VERIQTA





# VERSIONING, CLEANUP, AND RETENTION

## WHY ARTIFACT VERSIONS MATTER



- Every build is different.
- Versions allow you to track, reproduce, and deploy the exact artifact you tested.
- Essential for reliability, rollback, and compliance.

## IMMUTABLE RELEASES



- Once a release artifact is published, it must NEVER be changed or overwritten.
- Guarantees traceability and integrity.
- Ensures the same artifact is deployed every time.

## SNAPSHOT BUILDS



SNAPSHOT

- Used during active development.
- Can be overwritten.
- Examples:

my-app-1.0.0-SNAPSHOT

my-app-1.0.0-20250518.123456-1

- Not recommended for production.

## RELEASE BUILDS



RELEASE

- Immutable and final.
- Never overwritten.
- Examples:

my-app-1.0.0

my-app-1.1.0

- Used for production deployments.

## OLD ARTIFACT ACCUMULATION



- Every build creates new artifacts.
- Without cleanup, storage grows quickly.
- Leads to high storage costs and management difficulties.

### RISKS OF ACCUMULATION

- ⚠ Wasted storage
- ⚠ Slower artifact searches
- ⚠ Harder to manage
- ⚠ Higher infrastructure cost

## RETENTION POLICIES



- Define how long artifacts should be kept.
- Keep what you need.
- Delete what you don't.
- Balance cost and compliance.

## CLEANUP CONCEPTS



**1. IDENTIFY**  
Find old or unused artifacts using search and filters.

**2. DEFINE RULES**  
Set age, count, package type, or build info rules.

**3. DRY RUN**  
Preview which artifacts will be deleted.

**4. EXECUTE**  
Run the cleanup safely based on your rules.

**5. MONITOR**  
Review results and adjust policies as needed.

## WHY DELETING CARELESSLY CAN BREAK DEPLOYMENTS



- Old versions may still be used by running systems.
- Rollbacks may fail if the previous artifact is gone.
- Dependent artifacts may be required by other builds.
- Can cause failed deployments, downtime, and incidents.

## BEST PRACTICES



Use semantic versioning.



Keep releases immutable.



Keep enough old versions for rollback and audit.



Automate retention and cleanup policies.



Communicate cleanup rules with your team.



Always test cleanup rules before execution.





# BEGINNER TROUBLESHOOTING

## COMMON ISSUES AND HOW TO FIX THEM



### 401 UNAUTHORIZED

- Authentication failed.
- Credentials are missing or invalid.

**CHECK:** Token, username, password, permissions.

**FIX:** Use valid credentials with correct scopes.



### 403 FORBIDDEN

- Authenticated, but you don't have permission.
- Access is denied.

**CHECK:** User/group permissions for the repository.

**FIX:** Grant the required permissions.



### 404 ARTIFACT NOT FOUND

- Artifact path or name is wrong.
- Artifact does not exist.

**CHECK:** Repository, path, filename, and version.

**FIX:** Verify the path and ensure the artifact was published.



### FAILED UPLOADS

- Network issues.
- Insufficient permissions.
- Repository settings.

**CHECK:** Network, repository permissions, and quota.

**FIX:** Check connection, permissions, and repository configuration.



### FAILED DOWNLOADS

- Network issues.
- Wrong credentials.
- Permission denied.

**CHECK:** Connectivity, credentials, and repository access.

**FIX:** Verify access, token, and repository permissions.



### INCORRECT REPOSITORY URL

- Wrong URL or endpoint.
- Missing / wrong repository key.

**CHECK:** URL, port, and repository path.

**FIX:** Use the correct Artifactory URL and repository path.



### WRONG REPOSITORY TYPE

- Using the wrong repo type for your package.
- Example: Using Maven repo for npm packages.

**CHECK:** Repository type vs. package type.

**FIX:** Create or use the correct repository type.



### DOCKER AUTHENTICATION FAILURES

- Docker login failed.
- Wrong credentials or token.
- Insufficient Docker permissions.

**CHECK:** Docker login, token scopes, and registry permissions.

**FIX:** Re-login, use a valid token, and ensure correct permissions.



### DEPENDENCY RESOLUTION FAILURES

- Cannot find dependencies.
- Wrong repository configuration.
- Remote repo not working.

**CHECK:** Repository order, remote repository status, and proxies.

**FIX:** Adjust repository order and verify remote sources.

## A SIMPLE TROUBLESHOOTING SEQUENCE



**1. IDENTIFY**  
Identify the error message and what failed.



**2. GATHER INFO**  
Check logs, error details, and timestamps.



**3. CHECK BASICS**  
Validate URL, repo, credentials, and permissions.



**4. ISOLATE**  
Test with simple commands or a small file.



**5. FIX THE ISSUE**  
Apply the correct fix based on the root cause.



**6. VERIFY**  
Test again and confirm it is resolved.

## BEST PRACTICES



Use strong authentication and tokens.



Follow least privilege access principles.



Keep repository names and URLs consistent.



Monitor and clean artifacts with retention policies.



Always check logs before making changes.



Test changes in a non-production environment.

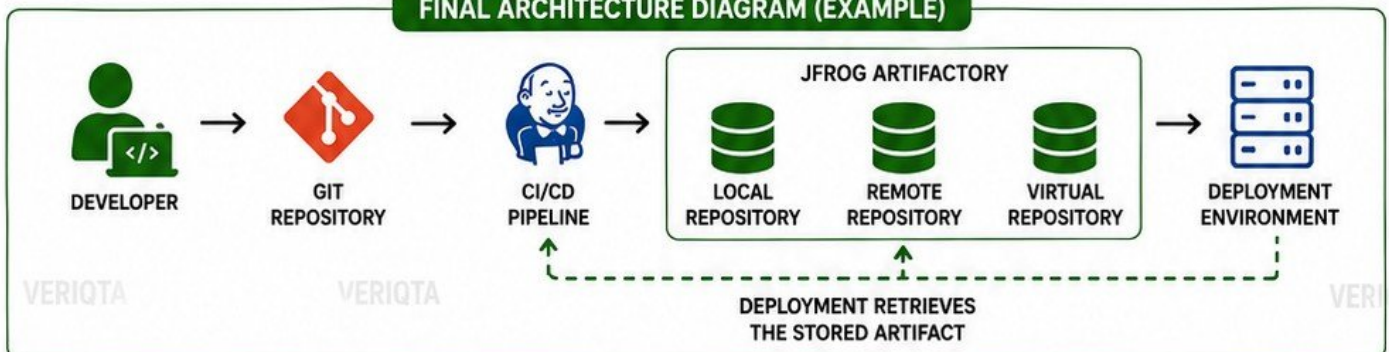


# COMPLETE BEGINNER PROJECT

## BUILD A SMALL END-TO-END ARTIFACT WORKFLOW

- 1**  **CREATE AN APPLICATION**  
Build a simple application (any language).
- 2**  **STORE SOURCE CODE IN GIT**  
Initialize a Git repository and push your code.
- 3**  **BUILD THE APPLICATION**  
Compile/build your application using the chosen build tool.
- 4**  **GENERATE A VERSIONED ARTIFACT**  
Package the build output and assign a version (e.g., 1.0.0).
- 5**  **CREATE THE REQUIRED ARTIFACTORY REPOSITORIES**  
Create local/remote/virtual repositories as needed.
- 6**  **CONFIGURE AUTHENTICATION**  
Create users or tokens and configure access securely.
- 7**  **UPLOAD THE ARTIFACT**  
Upload the artifact to Artifactory.
- 8**  **VERIFY IT IN ARTIFACTORY**  
Search and confirm the artifact is stored correctly.
- 9**  **DOWNLOAD THE SAME ARTIFACT**  
Download the artifact and verify its integrity.
- 10**  **INTEGRATE ARTIFACTORY WITH A CI/CD PIPELINE**  
Connect your pipeline to build, test, and publish artifacts.
- 11**  **DEPLOY THE STORED ARTIFACT**  
Deploy the artifact from Artifactory to your environment.
- 12**  **CONFIRM THE DEPLOYED VERSION**  
Validate the application version running in the environment.
- 13**  **TROUBLESHOOT ONE INTENTIONALLY BROKEN PIPELINE**  
Break the pipeline intentionally, identify the issue, and fix it.
- 14**  **FINAL ARCHITECTURE DIAGRAM**  
Draw your end-to-end architecture and explain each component.

### FINAL ARCHITECTURE DIAGRAM (EXAMPLE)



#### KEY TAKEAWAY

You learned the complete journey of an artifact from code to production using JFrog Artifactory. This ensures consistency, reliability, and security at every step.



#### WHAT YOU ACHIEVED

- ✓ Built and stored a versioned artifact
- ✓ Integrated Artifactory with CI/CD
- ✓ Deployed the stored artifact successfully
- ✓ Troubleshoot and fixed a pipeline issue